

Retrieval Augmented Generation(RAG) with Azure AI Search

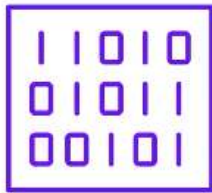
➔ Introduction to Azure AI search

- Azure AI Search
 - Deliver high-quality responses with a vector database built for advanced retrieval augmented generation (RAG) and modern search
 - grounding data for conversational and copilot search. When you create a search service, you work with the following capabilities:
 - A search engine for vector search and fall at and hybrid search over a search index
 - Rich indexing with integrated data chunking and vectorization, lexical analysis for text, and optional applied AI for content extraction and transformation
 - Rich query syntax for vector queries, text search, hybrid queries, fuzzy search, autocomplete, geo-search and other
 - Relevance and query performance tuning with semantic ranking, scoring profiles, quantization for vector queries and parameters for controlling query behaviors at runtime
 - Azure scale, security, and reach
 - Abundant integration at the data layer, machine learning layer, Azure AI services and Azure OpenAI
 - How to use Azure AI Search:
 - Scenario:
 - You are a software engineer in a company PeerBNB where you provide short term rental place around the world
 - The people who stay leave their reviews on website, so company wants those customers to see the recommendations on the basis of the reviews
 - Task : Map the intent from user query and map it to high quality data and return high quality response back to user
 - What we need to do : We have to take all the users data in such a way that if the user comes search they

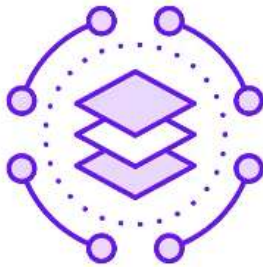
will get exactly what they were looking for , even if there is no keyword matching

➔ Steps:

Steps



Data Preparation



Index Creation



Add Documents



Query Index

➔ Implementing RAG for Hotel Reviews

- Steps :
 - Preparing Data for Indexing
 - Creating Search Service on Azure
 - Creating Search Index
 - Querying Search Index
 - Searching Index with Semantic Reranker
- Github Link : github.com/hhimanshuips-reg-azure-di-search
- Dataset : Hotel reviews dataset from kagel

➔ Preparing Data for Indexing:

1. Step 1 : convert CSV to Json files

➔ Prepare data for Azure AI indexing

1. Step 2 : Create Embeddings for each of the json files

```
1 import os
2 import json
3
4 transformed_dir = "../data/transformed"
5 embedded_dir = "../data/embedded"
6
7 if not os.path.exists(embedded_dir):
8     os.makedirs(embedded_dir)
9
10 def prepare_embedding_str(review_json):
11     return f"REVIEW_TITLE: {review_json['review_title']} REVIEW_TEXT: {review_json['review_text']} HOTEL_NAME: {review_json['hotel_name']} HOTEL_CITY: {review_json['city']} HOTEL_STATE: {review_json['hotel_state']}"
12
13 client = OpenAI()
14 for file in os.listdir(transformed_dir):
15     with open(f"{transformed_dir}/{file}", "r") as f:
16         review = json.load(f)
17
18         ## start here
19
20         with open(f"{embedded_dir}/{file}", "w") as f:
21             json.dump(review, f, indent=2)
```

- File Handling: The code imports `os` and `json` to navigate directories and read/write JSON files .
- Data Preparation: The `prepare_embedding_str` function formats the JSON review data into a single string combining the title, text, hotel name, city, and state .
- Loop Processing: A for loop iterates through files in the `transformed_dir`, reads each JSON file, and ostensibly processes it .
- Output: The code opens files in `embedded_dir` to write data, though the current snippet saves the original JSON object rather than a generated embedding.

➔ Creating the Search Index:

- Open portal.azure.com/#home
- Then create your account on azure AI platform
- Then create a resource
- Create a azure ai search instance
- Add all the details and deploy

➔ Adding Document to search index:

- Before adding document we need to define schema

- Adding code snippet for adding schema with certain fields

```

17 vector_search_profile_name = f'{index_name}_profile'
18
19 fields = [
20     SimpleField(name="id", type=SearchFieldDataType.String, key=True),
21     SearchableField(name="review_text", type=SearchFieldDataType.String, analyzer_name="en.lucene"),
22     SearchableField(name="review_title", type=SearchFieldDataType.String, analyzer_name="en.lucene"),
23     SimpleField(name="dateAdded", type=SearchFieldDataType.DateTimeOffset, filterable=True, sortable=True),
24     SimpleField(name="city", type=SearchFieldDataType.String, filterable=True),
25     SimpleField(name="hotel_name", type=SearchFieldDataType.String, filterable=True),
26     SimpleField(name="hotel_state", type=SearchFieldDataType.String, filterable=True),
27     SearchField(
28         name="embedding",
29         type=SearchFieldDataType.Collection(SearchFieldDataType.Single),
30         vector_search_dimensions=1536, # Adjust this to match your embedding size
31         vector_search_profile_name=vector_search_profile_name,
32     ),
33 ]
34
35 # Vector search configuration
36 hnsw_algorithm_name = f'{index_name}_hnsw'
37 vector_search = VectorSearch(
38     algorithms=[HnswAlgorithmConfiguration(name=hnsw_algorithm_name)],
39     profiles=[
40         VectorSearchProfile(
41             name=vector_search_profile_name,
42             algorithm_configuration_name=hnsw_algorithm_name,
43         )
44     ],
45 )

```

- Before creating search index in azure we need to connect an link to azure ai search

```

7 load_dotenv()
8
9 {variable} service_name: str | None
10 service_name = os.getenv("SERVICE_NAME")
11 admin_key = os.getenv("SEARCH_ADMIN_KEY")
12 index_name = os.getenv("SEARCH_INDEX_NAME")
13
14 # Initialize the search index client
15 endpoint = f"https://{service_name}.search.windows.net/"
16 credential = AzureKeyCredential(admin_key)
17 index_client = SearchIndexClient(endpoint=endpoint, creden

```

-
- Checking and uploading all the doc files on azure portal

```

18 data['embedding'] = [float(x) for x in data['embedding']]
19
20 return data
21
22 # Directory containing the JSON files
23 directory = '../data/embedded'
24
25 # List to store all documents
26 documents = []
27
28 # Iterate through all JSON files in the directory
29 for filename in os.listdir(directory):
30     if filename.endswith('.json'):
31         print(f"Processing {filename}")
32         file_path = os.path.join(directory, filename)
33         document = process_json_file(file_path)
34         documents.append(document)
35
36 # If we have 1000 documents, upload them in batch
37 if len(documents) == 1000:
38     result = search_client.upload_documents(documents)
39     print(f"Uploaded {len(result)} documents")
40     documents = [] # Clear the list for the next batch
41
42 # Upload any remaining documents

```

➔ Querying the search index:

- As the data is now connected with Azure ai search
- We can write queries
- We need to embed user query by using the same embedding model

```

1 from openai import OpenAI
2
3 openai_client = OpenAI()
4
5 def embed_query(query: str) -> List:
6     response = openai_client.embeddings.create(
7         input=query, model="text-embedding-3-small"
8     )
9     return response.data[0].embedding

```

- Then creating three types of search query:

```

3 def full_text_search(query, top=5):
4     """Perform a full-text search."""
5     * results = search_client.search(search_text=query, top=top)
6     return list(results)
7
8 def vector_search(query):
9     embedded_query = embed_query(query)
10    vector_query = VectorizedQuery(vector=embedded_query, k_nearest_neighbors=3, fields="embedding")
11
12    results = search_client.search(
13        search_text=None,
14        vector_queries=[vector_query],
15        select=["hotel_name", "review_text", "review_title", "city"],
16    )
17    return list(results)
18
19 def hybrid_search(query, top=5):
20     """Perform a hybrid search (combination of full-text and vector search)."""
21     embedded_query = embed_query(query)
22     vector_query = VectorizedQuery(vector=embedded_query, k_nearest_neighbors=3, fields="embedding")
23     results = search_client.search(
24         search_text=query,
25         vector_queries=[vector_query],
26         select=["id", "review_text", "review_title", "hotel_name", "city"],
27         top=top,
28     )

```

▪ 1. Full-Text Search

- The full_text_search function performs a traditional keyword-based search. It uses the search_text parameter to find exact or partial matches within the indexed text.

▪ 2. Vector Search

- _search function performs a semantic search.
- **Embedding:** It first converts the text query into a numerical vector using embed_query(query).
- **Nearest Neighbors:** It searches for the top 3 (k_nearest_neighbors=3) documents whose embeddings are mathematically closest to the query vector in the "embedding" field.
- **Selection:** It specifically retrieves the fields "hotel_name", "review_text", "review_title", and "city".

▪ 3. Hybrid Search

- The `hybrid_search` function combines both methods. By providing both `search_text` and `vector_queries` to the search method, it leverages keyword matching and semantic meaning simultaneously to improve result relevance.

➔ Searching Index with Semantic Reranker

- It is a paid feature
- What is semantic ranking?
 - Semantic ranker is a collection of query-side capabilities that improve the quality of an index 125-ranked or B ranter search result for text-based queries. When you enable it on your search service, semantic ranking extends the query execution pipeline in two ways
 - First it adds secondary ranking over an initial result set that was scored using BM25 or RRF. This secondary ranking uses multi-lingual, deep learning models adapted from Microsoft Bing to promote the most semantically relevant results
 - Second, it extracts and returns captions and answers in the response, which you can render on a search.

```

50
51 semantic_config = SemanticConfiguration(
52     name="ps-hotels-semantic-config",
53     prioritized_fields=SemanticPrioritizedFields(
54         title_field=SemanticField(field_name="review_title"),
55         content_fields=[SemanticField(field_name="review_text")],
56         keywords_fields=[
57             SemanticField(field_name="city"),
58             SemanticField(field_name="hotel_name"),
59             SemanticField(field_name="hotel_state")
60         ]
61     )
62 )
63
64 # Create the semantic settings with the configuration
65 semantic_search = SemanticSearchConfigurations([semantic_config])
66
67 index = SearchIndex(name=index_name, fields=fields, vector_search=vector_search, semantic_search=semantic_search)

```

-
- Sets up Semantic Ranking (formerly Semantic Search) to improve relevance by understanding the intent of the query.
 - `prioritized_fields`: Tells the engine which fields are most important for context. In this case:
 - Title: `review_title`
 - Content: `review_text`
 - Keywords: `city`, `hotel_name`, and `hotel_state`
- Index Creation:
 - Combines the vector configuration and the semantic settings into a single `SearchIndex` object.
 - This index is now ready to be deployed to an Azure AI Search service to handle hybrid queries (combining keyword, vector, and semantic re-ranking)

- Adding the code snippet for adding reranking:

```
query_type=QueryType.SEMANTIC,  
semantic_configuration_name="ps-hotels-semantic-config",  
query_caption=QueryCaptionType.EXTRACTIVE,  
query_answer=QueryAnswerType.EXTRACTIVE,  
top=top,
```